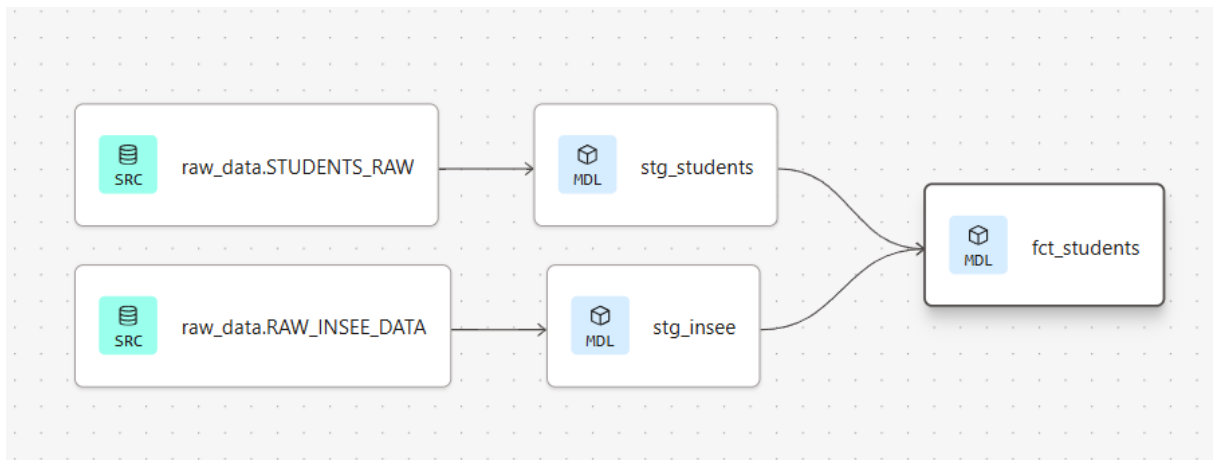


Section 1 : Architecture du Workflow



Explication de l'architecture :

Ce graphe de dépendance (Lineage Graph) généré par dbt illustre le flux de transformation des données au sein de notre entrepôt Snowflake :

- Les Sources (raw_data) : À gauche, les tables STUDENTS_RAW (données internes d'OpenClassrooms) et RAW_INSEE_DATA (données démographiques externes) représentent les données brutes telles qu'elles ont été ingérées dans le schéma RAW de Snowflake.
- La couche Staging (stg_) : C'est la première étape de transformation. Les modèles stg_students et stg_insee ont pour rôle de nettoyer et standardiser la donnée brute. C'est ici qu'ont été appliqués les filtres de base, le renommage des colonnes, le dédoublonnage, ainsi que l'anonymisation des identifiants utilisateurs pour garantir une stricte conformité RGPD.
- La couche Marts (fct_) : À droite, le modèle final fct_students représente la table finale. Il consolide les données en croisant la table des étudiants avec celle de la population de l'INSEE (jointure sur la région et l'année). C'est dans ce modèle que toute la logique d'analyse et les calculs sont effectués.

Section 2 : Modèles SQL et Règles Métier

Cette section détaille le code de transformation écrit dans dbt. L'objectif de ces modèles est de nettoyer, sécuriser et enrichir la donnée brute pour la rendre exploitable par les outils d'analyse, tout en garantissant la reproductibilité du pipeline.

1. Modèle de Staging : stg_students.sql (Données Openclassrooms)

```
stg_students.sql

dbt-cloud-70471823537764-projet-oc-data > models > staging > stg_students.sql

1  with source as (select * from {{ source("raw_data", "STUDENTS_RAW") }})
2
3  select
4      -- RGPD : Anonymisation de l'identifiant
5      sha2(user_id) as student_id_hashed,
6
7      -- Infos sociodémographiques
8      trim(gender) as gender,
9      trim(age_group) as age_group,
10     trim(region) as region,
11
12     cast(year_path_started as int) as start_year,
13
14     path_category_name as path_category
15 from source
16 -- Filtre sur la filière Data pour la mission
17 where path_category_name ilike '%Data%'
18 -- Dédoublonnage : garde l'inscription la plus récente pour chaque étudiant
19 qualify row_number() over (partition by user_id order by year_path_started desc) = 1
20
```

Ce premier modèle prépare les données étudiantes d'OpenClassrooms. Plusieurs règles critiques y sont appliquées :

- Conformité RGPD : L'identifiant utilisateur (user_id) est immédiatement haché grâce à la fonction SHA2. Cela permet de suivre un étudiant de manière unique sans jamais exposer ses données personnelles.
- Typage et Nettoyage : Les espaces superflus sont supprimés (TRIM) et l'année d'inscription est forcée au format entier (CAST ... AS INT) pour permettre les calculs ultérieurs.
- Filtre parcours : Conformément à la demande de la direction, la base est restreinte aux seuls parcours "Data" via un filtre ILIKE.
- Qualité de la donnée (Dédoublonnage) : Utilisation de la fonction avancée QUALIFY ROW_NUMBER() pour gérer les doublons identifiés dans la source brute, en ne conservant que l'inscription la plus récente par étudiant.

2. Modèle de Staging : stg_insee.sql (Données Externes)

```
stg_insee.sql X
dbt-cloud-70471823537764-projet-oc-data > models > staging > stg_insee.sql

1  WITH source AS (
2      SELECT * FROM {{ source('raw_data', 'RAW_INSEE_DATA') }}
3  )
4
5  SELECT
6      TRIM(REGION) AS region_name,
7      TRIM(SEXE) AS gender,
8      TRIM(TRANCHE_AGE) AS age_group,
9      POPULATION AS population,
10     ANNEE AS year
11 FROM source
```

Ce modèle standardise les données démographiques issues de l'INSEE (préalablement restructurées).

- Standardisation : Les noms de colonnes sont traduits en anglais (region_name, gender, age_group) pour s'aligner parfaitement avec les conventions de nommage du modèle de données interne
- Nettoyage : La fonction TRIM est appliquée pour éviter que des espaces invisibles ne fassent échouer la jointure finale.

3. Modèle Final : fct_students.sql (Le Data Mart)

```
fct_students.sql x settings.json
dbt-cloud-70471823537764-projet-oc-data > models > marts > fct_students.sql

1  -- 1. Récupération des étudiants dédoublonnés (Filière Data uniquement)
2  WITH students AS (
3      SELECT * FROM {{ ref('stg_students') }}
4  ),
5
6  -- 2. Récupération des données INSEE nettoyées (Population Totale)
7  insee_totaux AS (
8      SELECT
9          CASE
10             WHEN region_name = 'Centre-Val-de-Loire' THEN 'Centre-Val de Loire'
11             ELSE region_name
12          END AS region_name_clean,
13          year,
14          population AS total_population_region
15      FROM {{ ref('stg_insee') }}
16      WHERE gender = 'Ensemble' AND age_group = 'Total'
17  ),
18
19  -- 3. Calcul des indicateurs de base et jointure
20  base_analysis AS (
21      SELECT
22          s.*,
23          i.total_population_region,
24          -- Nombre total d'étudiants par région et par an (pour les calculs de % après)
25          COUNT(*) OVER (PARTITION BY s.region, s.start_year) AS total_students_region_year,
26          -- Nombre total d'étudiants par an (pour l'évolution globale)
27          COUNT(*) OVER (PARTITION BY s.start_year) AS total_students_per_year
28      FROM students s
29      LEFT JOIN insee_totaux i
30          ON s.region = i.region_name_clean
31          AND s.start_year = i.year
32  )
33
34  -- 4. Calcul des indicateurs avancés (Prêts pour Excel)
35  SELECT
36      *,
37      -- INDICATEUR 1 : Taux de pénétration (Nb étudiants pour 100k habitants)
38      (total_students_region_year / total_population_region) * 100000 AS penetration_rate_100k,
39
40      -- INDICATEUR 2 : Part du genre par année (Pour mesurer l'évolution de la parité)
41      -- Calcule combien de % représente mon genre (M/F) par rapport au total de l'année
42      ROUND(
43          COUNT(*) OVER (PARTITION BY start_year, gender) * 100.0 / total_students_per_year,
44          2
45      ) AS gender_yearly_share_pct,
46
47      -- INDICATEUR 3 : Part de la tranche d'âge (Pour le profil sociodémographique)
48      ROUND(
49          COUNT(*) OVER (PARTITION BY age_group) * 100.0 / COUNT(*) OVER (),
50          2
51      ) AS age_group_global_share_pct,
52
53      -- INDICATEUR 4 : Indice de maturité (Est-ce un profil en reconversion ?)
54      CASE
55          WHEN age_group IN ('35-39 ans', '40-44 ans', '45-49 ans', '50-54 ans', '55-59 ans', '60 ans ou plus') THEN 'Reconversion (35+)'
56          ELSE 'Jeune profil (<35)'
57      END AS career_stage
58
59  FROM base_analysis
```

Ce modèle métier final est le cœur de l'analyse. Il consolide toutes les sources et pré-calculer les indicateurs de performance (KPIs) :

- Gestion de la fiabilité : Pour la jointure avec l'INSEE, le nom de la région "Centre-Val-de-Loire" est corrigé pour correspondre à la casse du fichier étudiant. De plus, la population INSEE est strictement filtrée sur Ensemble et Total pour éviter tout double comptage démographique.
- Jointure (LEFT JOIN) : L'enrichissement se fait sur une double clé temporelle et spatiale (region et start_year).
- Ingénierie des features (Window Functions) : Au lieu de déléguer les calculs à l'outil de visualisation, les indicateurs clés sont calculés directement en SQL via des fonctions de fenêtrage (OVER PARTITION BY). Le modèle génère ainsi :
 - o Le taux de pénétration (étudiants pour 100 000 habitants).
 - o L'évolution annuelle de la parité (gender_yearly_share_pct).
 - o Un axe d'analyse métier sur mesure classifiant les étudiants en profils "Reconversion (35+)" ou "Jeune profil".

Section 3 : Contrôle Qualité et Tests

1. Définition des règles de fiabilité :

Pour garantir l'intégrité des données, des tests automatisés ont été déclarés dans les fichiers de configuration YAML.

Un test *not_null* sécurise la donnée entrante sur la source brute (STUDENTS_RAW).

Des tests *unique* et *not_null* sont appliqués sur la table finale (fct_students) pour s'assurer qu'aucun doublon ni aucune valeur manquante (générée lors des jointures) ne viennent fausser les calculs de l'analyse.

```
schema.yml X
dbt-cloud-70471823537764-projet-oc-data > models > marts > schema.yml
DbtPropertiesFile (dbt_yaml_files-latest-fusion.json)
1 version: 2
2
3 models:
4   - name: fct_students
5     description: "Table finale enrichie avec les données de population INSEE."
6     columns:
7       - name: student_id_hashed
8         description: "ID anonymisé de l'étudiant."
9     tests:
10       - unique
11       - not_null
```

```
_sources.yml X
dbt-cloud-70471823537764-projet-oc-data > models > staging > _sources.yml
DbtPropertiesFile (dbt_yaml_files-latest-fusion.json)
1 version: 2
2
3 sources:
4   - name: raw_data
5     database: OC_STUDENTS_PROJ
6     schema: RAW
7     tables:
8       Generate model
9       - name: STUDENTS_RAW
10         description: "Données brutes des étudiants."
11         # tests
12         columns:
13           - name: USER_ID
14             tests:
15               - not_null
16       Generate model
17       - name: RAW_INSEE_DATA
18         description: "Données démographiques de l'INSEE nettoyées."
```

2. Validation du pipeline :

L'exécution de la commande `dbt test` confirme le succès absolu de l'ensemble des contrôles (3 tests validés, 0 erreur). Cela prouve techniquement que la logique de dédoublonnage a parfaitement fonctionné et que le Master Dataset exporté pour la visualisation est 100 % fiable.

dbt test ✔ SuccessCancelRerun

 main

 4 days ago

> System logs

All **3**Pass **3**Warn **0**Error **0**Skip **0**Running **0**

>	✔ not_null_fct_students_student_id_hashed	0.85s
>	✔ source_not_null_raw_data_STUDENTS_RAW_USER_ID	1.29s
>	✔ unique_fct_students_student_id_hashed	1.49s